

# DrivAerML Dataset Splits

Deterministic train/validation/test splits for the [DrivAerML](#) dataset [1]. The split manifest is stored at `splits/manifest.json` as a flat JSON object with keys named `{split_name}_{train,val,test}` and case IDs that match the on-disk run directories.

DrivAerML contains 500 vehicle geometry variants at a fixed operating condition. The public force/moment table has 484 rows; the missing 16 runs are unavailable author-held-back cases and are excluded from all public train/validation/test splits. The geometry-parameter table contains all 500 design rows and is used for nested data-efficiency subset selection and image-score imputation. The `geometry` split itself uses sampled STL-surface Chamfer distances to measure how different each public geometry is from its neighbors.

## Splits at a glance

| Split                        | Type    | Train | Val | Test | What it tests                                                                |
|------------------------------|---------|-------|-----|------|------------------------------------------------------------------------------|
| <code>full</code>            | In-dist | 400   | 34  | 50   | Seed-42 random public baseline split                                         |
| <code>medium</code>          | In-dist | 133   | 34  | 50   | Data efficiency, 1/3 of <code>full</code> training data                      |
| <code>scarce</code>          | In-dist | 67    | 34  | 50   | Data efficiency, 1/6 of <code>full</code> training data                      |
| <code>super_scarce</code>    | In-dist | 11    | 34  | 50   | Extreme data efficiency, 1/36 of <code>full</code> training data             |
| <code>geometry</code>        | OOD     | 339   | 48  | 97   | STL-surface Chamfer extrapolation using the top 20% local-isolation distance |
| <code>high_drag</code>       | OOD     | 339   | 48  | 97   | High-drag extrapolation using the top 20% <code>cd</code>                    |
| <code>low_drag</code>        | OOD     | 339   | 48  | 97   | Low-drag extrapolation using the bottom 20% <code>cd</code>                  |
| <code>rear_separation</code> | OOD     | 339   | 48  | 97   | Image-derived low-speed wake and rear-separation extent                      |

### Difficulty ladders:

- Data efficiency: `full` < `medium` < `scarce` < `super_scarce`, with fixed validation/test sets.
- OOD physics: `geometry`, `high_drag`, `low_drag`, and `rear_separation`.

## Which split should I use?

- Simple baseline or literature compatibility: `full`
- Data efficiency study: compare `super_scarce`, `scarce`, `medium`, and `full`
- Geometry extrapolation: `geometry`
- Image-observed rear-separation regimes: `rear_separation`
- Targeted coefficient extrapolation: `high_drag` or `low_drag`

## Using the committed splits

For standard benchmark use, consume the committed manifest at `splits/manifest.json`; no regeneration is required. The manifest is the source of truth for all split membership and contains one `train`, `validation`, and `test` key for each split listed above.

```
import json
from pathlib import Path
```

```
manifest = json.loads(Path("splits/manifest.json").read_text())

train_ids = manifest["geometry_train"]
val_ids = manifest["geometry_val"]
test_ids = manifest["geometry_test"]
```

Change the `geometry` prefix to `full`, `medium`, `scarce`, `super_scarce`, `high_drag`, `low_drag`, or `rear_separation` to select a different split. Each value is a sorted list of `run_N` directory names. The manifest already excludes the 16 unavailable held-back cases, so users should not filter those IDs again unless their local dataset copy is incomplete.

## Design principles

- **Validation is always in-distribution with train.** For OOD splits, the validation set is drawn from the training-side population, not from the extreme OOD test region.
- **Keep the public baseline stable.** `full` is a seed-42 random public split: 400 train, 34 validation, 50 test, with the 16 unavailable cases excluded.
- **Use direct surface distances for geometry OOD.** `geometry` ranks public STL surfaces by nearest-neighbor Chamfer isolation, which is a stronger geometry-difference signal than parameter-space distance alone.
- **Use the dataset metadata directly where it defines the split axis.** Force-regime splits are generated from `force_mom_all.csv`. The geometry split is generated from STL-surface Chamfer scores in `chamfer_metrics.csv`; `geo_parameters_all.csv` supports the nested data-efficiency subsets and image-score imputation.
- **Use flow images as physics labels.** The retained image-derived split scores low-speed recirculation/separation extent from centreline and near-rear velocity-magnitude PNGs.
- **Represent DrivAerML physics.** The retained OOD axes target surface-shape novelty, high- and low-drag regimes, and visible rear-separation flow structure.
- **Nested data-efficiency subsets.** `super_scarce_train` is a strict subset of `scarce_train`, `scarce_train` is a strict subset of `medium_train`, and `medium_train` is a strict subset of `full_train`.
- **Test set integrity.** Test cases should not be used for hyperparameter tuning, model selection, normalization fitting, or visual inspection-driven iteration.

## Split details

### full

The default public split is a seeded random split over run IDs, not a physics-stratified split. It constructs `torch.randperm(500)` with seed 42, shifts the IDs to 1..500, removes the 16 unavailable hidden cases, then assigns the first 400 public IDs to train, the next 50 to test, and the remaining 34 to validation. The case IDs are sorted before being stored in the manifest. For reference, these IDs match the public [DrivAerMLDefaultSplitIDs](#) implementation in Noether [3].

### medium, scarce, and super\_scarce

Same validation/test as `full`, but the training set is a nested force- and geometry-diverse subset of `full_train`: 133 cases for `medium`, 67 cases for `scarce`, and 11 cases for `super_scarce`. The subset order is a greedy max-min selection in standardized force/geometry feature space using `cd`, `cl`, `cs`, and all public geometry parameters.

By construction, `super_scarce_train` is a subset of `scarce_train`, `scarce_train` is a subset of `medium_train`, and `medium_train` is a subset of `full_train`.

## geometry

The preferred geometry OOD split when STL-derived geometry metrics are available. It uses `chamfer_metrics.csv`, where each public run is represented by deterministic surface samples from its STL mesh. The committed metrics were generated with 10,000 surface samples per public run, seed 42, no recentering, global median bounding-box scaling, and symmetric Chamfer RMS distances.

The split uses `ood_score`, currently the mean distance to the 10 nearest neighboring public geometries:

```
geometry_score_i = mean_10_nearest_neighbors(chamfer_distance_i)
```

The top 20% of public runs by this local-isolation score form the OOD test set. Validation is sampled from the remaining training-side pool:

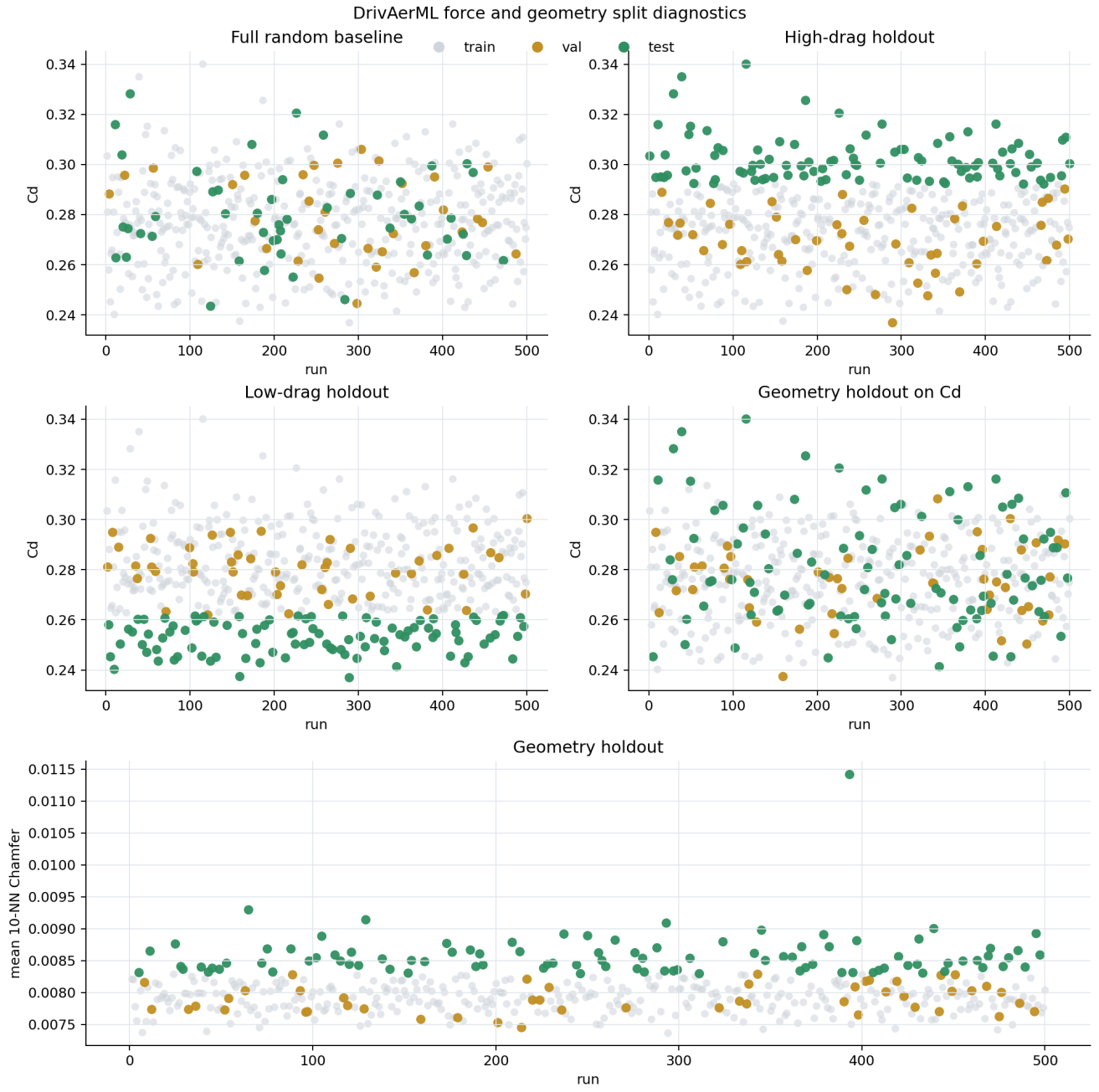
```
geometry_test  = top_20_percent_i(geometry_score_i)
geometry_pool  = public_runs - geometry_test
geometry_val   = deterministic_random_sample(geometry_pool, round(0.125 * |geometry_pool|))
geometry_train = geometry_pool - geometry_val
```

This split captures surface-level differences that may not dominate standardized parameter distances. In the current Chamfer metrics, the most locally isolated public geometries include `run_393`, `run_65`, `run_129`, `run_293`, `run_439`, `run_345`, `run_495`, and `run_237`. Figure 1 shows the geometry split both against Cd and against the Chamfer score used to define the holdout, and Figure 2 shows complete-car examples from the low-score training side and high-score geometry-test side, including a same-crop transparent overlay.

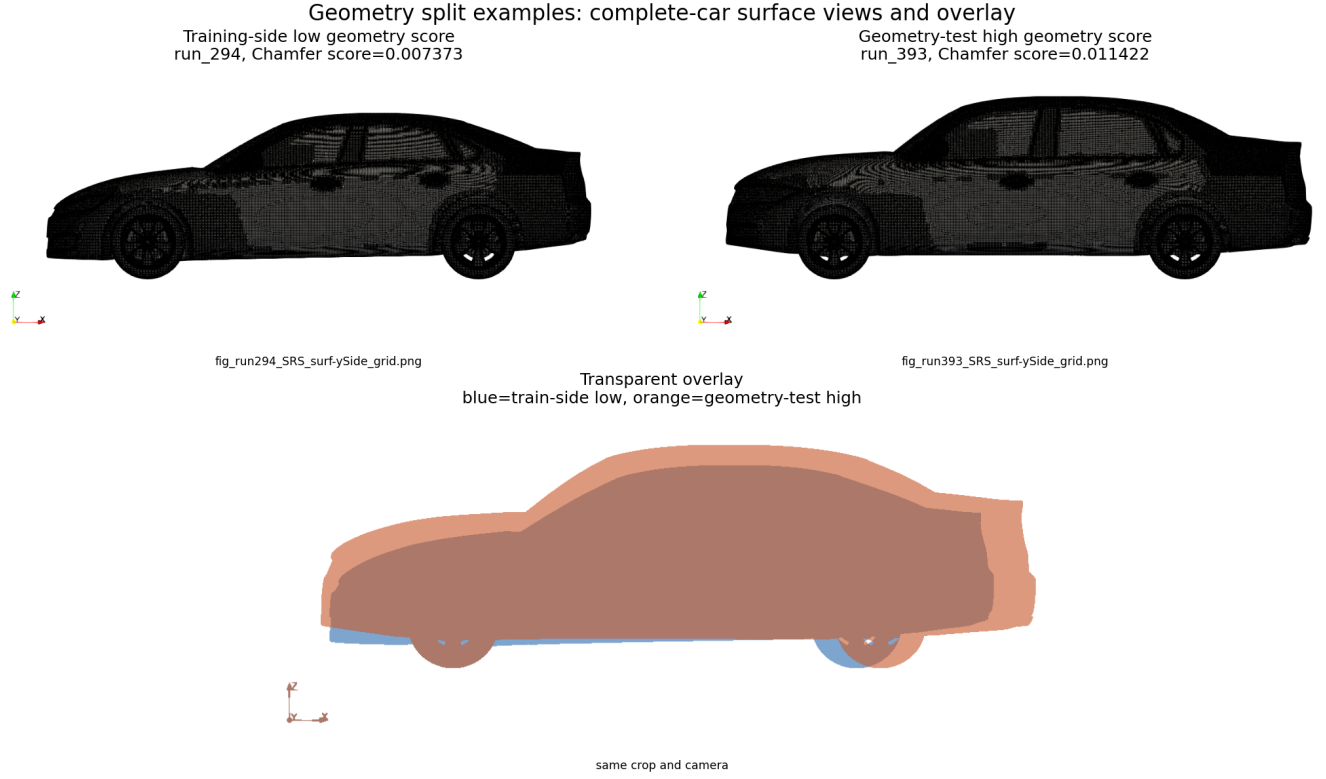
## high\_drag and low\_drag

Both splits sort public cases by drag coefficient `cd`. `high_drag` holds out the top 20% by `cd`; `low_drag` holds out the bottom 20% by `cd`. In both cases, train/validation are sampled from the complementary 80% so validation remains in-distribution with training. Figure 1 shows these holdouts across run IDs.

In the current CSV, the highest-drag public runs include `run_115`, `run_39`, `run_29`, `run_186`, and `run_226`. The lowest-drag public runs include `run_289`, `run_159`, `run_10`, `run_345`, and `run_426`.



**Figure 1:** Force and geometry split diagnostics. The panels show the full random baseline, high-drag holdout, low-drag holdout, geometry holdout on  $C_d$ , and geometry holdout on STL-surface Chamfer score. The x-axis is run ID, and points are colored by each split's train, validation, and test partitions.



**Figure 2:** Geometry split examples. The low-score training-side case is `run_294` (`geometry_score`=0.007373, `Cd`=0.281193, `C1`=-0.017952), and the high-score geometry-test case is `run_393` (`geometry_score`=0.011422, `Cd`=0.285732, `C1`=0.052227). The bottom panel overlays the same-crop side views with `run_294` in blue and `run_393` in orange.

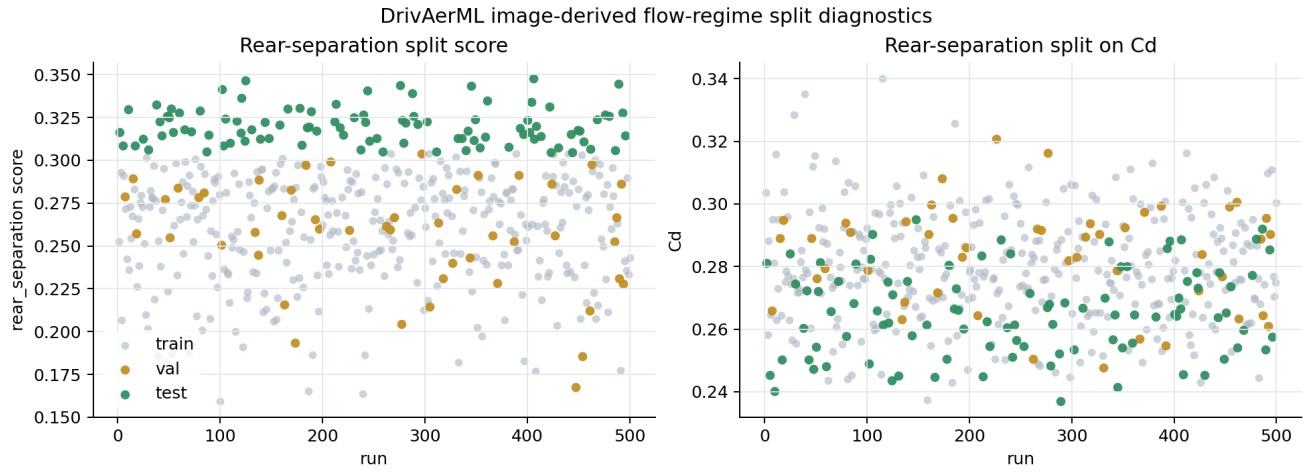
## Image-derived flow-regime split

The image-derived split is intended to capture a visible flow-physics regime that is not fully described by high or low integrated coefficients. It uses selected PNG diagnostics from each `run_*/images/` folder, and Figure 3 shows the resulting rear-separation score distribution plus the same train/validation/test membership against `Cd`:

- **rear\_separation:** low-speed wake area from the `y=0` centreline velocity-magnitude PNG and seven near-rear `xNormal` velocity-magnitude PNGs (p43000 through p55000).

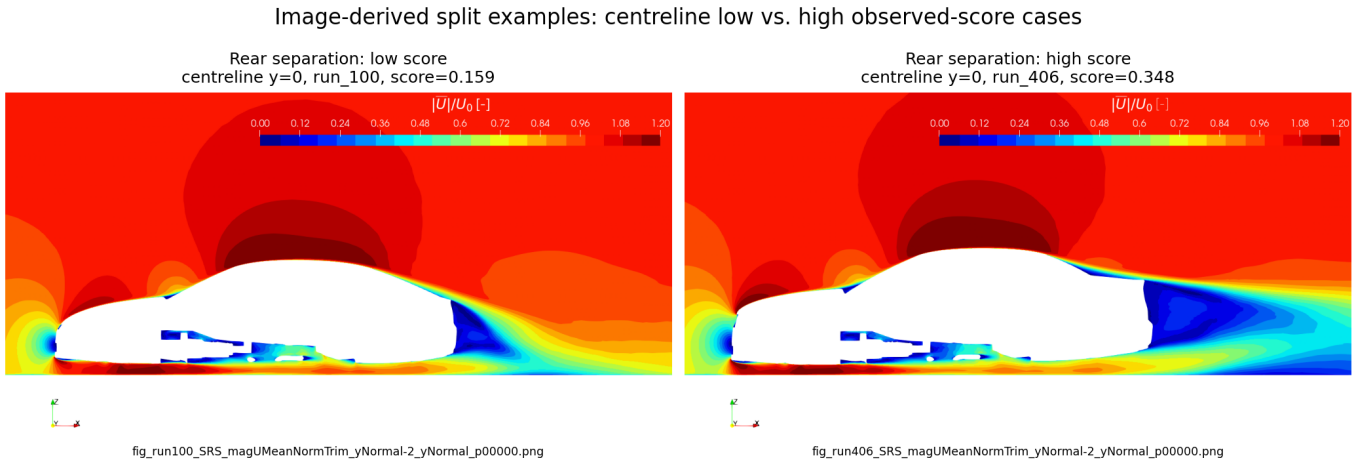
The score estimates recirculation/separation size by counting the low-speed blue/cyan/green part of the fixed `|U|/U0` colormap. The committed score combines 60% centreline wake area with 40% mean near-rear `xNormal` low-speed area.

For this score, the top 20% of public runs are held out as test. Validation is sampled from the remaining 80%.



**Figure 3:** Image-derived rear-separation split diagnostics. The left panel shows the rear-separation score used for the holdout, and the right panel shows  $C_d$  for the same `rear_separation_train`, `rear_separation_val`, and `rear_separation_test` partitions.

Figure 4 shows centreline  $y=0$  velocity-magnitude slices for observed low-score and high-score cases from the image-derived metric.



**Figure 4:** Centreline velocity-magnitude examples for low and high rear-separation scores. The low-score case is `run_100` ( $C_d=0.292211$ ,  $C_l=0.147656$ ), and the high-score case is `run_406` ( $C_d=0.266587$ ,  $C_l=0.019273$ ).

## Repeatability and transparency

The committed manifest is intended for normal use. The commands below are for auditing the split construction, recreating the figures, or refreshing the artifacts after changing the source data.

From a clean split-package checkout, download `force_mom_all.csv` and `geo_parameters_all.csv` from the Hugging Face dataset repo and then regenerate the manifest and diagnostic plots:

```
python3 scripts/download_hf_inputs.py --output-dir data
python3 scripts/generate_splits.py
python3 scripts/visualize_flow_regimes.py
python3 scripts/visualize_image_regimes.py
```

This exact path uses committed Chamfer geometry scores and committed rear-separation image scores. To render the example figures from source PNGs rather than placeholders, also run:

```
python3 scripts/download_hf_inputs.py --output-dir data --include-report-images
python3 scripts/visualize_geometry_examples.py
python3 scripts/visualize_split_examples.py
```

Full report regeneration then uses:

```
python3 scripts/generate_splits.py
python3 scripts/visualize_flow_regimes.py
python3 scripts/visualize_geometry_examples.py
python3 scripts/visualize_image_regimes.py
python3 scripts/visualize_split_examples.py
latexmk -pdf -cd docs/README.tex
```

For an existing dataset checkout, set `DRIVAERML_DATA_ROOT` to the directory containing `force_mom_all.csv` and `geo_parameters_all.csv`. If the PNGs live elsewhere, set `DRIVAERML_IMAGE_ROOT` to the directory containing `run_*/images/`.

To recompute `data/image_metrics.csv` from source images, use:

```
python3 scripts/download_hf_inputs.py --output-dir data --include-image-score-pngs
python3 scripts/generate_splits.py
```

To recompute the Chamfer source data from STL meshes, run the standalone surface-distance script in a work directory and refresh only the metrics CSV used by the split generator:

```
python3 scripts/download_hf_inputs.py --output-dir /tmp/drivaerml_inputs --include-stls
mkdir -p /tmp/drivaerml_chamfer
python3 scripts/compute_chamfer_splits.py \
  --data-root /tmp/drivaerml_inputs \
  --output-dir /tmp/drivaerml_chamfer \
  --samples 10000 \
  --workers 16 \
  --base-manifest splits/manifest.json
cp /tmp/drivaerml_chamfer/chamfer_metrics.csv data/chamfer_metrics.csv
```

The committed source artifacts were generated against a local checkout with:

```
<dataset-root>/force_mom_all.csv
<dataset-root>/geo_parameters_all.csv
<split-package-root>/data/chamfer_metrics.csv
<split-package-root>/splits/manifest.json
<image-root>/run_*/images/
```

## Manifest format

```
{
  "full_train": ["run_1", "run_2", "..."],
  "full_val": ["run_4", "..."],
  "full_test": ["run_11", "..."],
  "scarce_train": ["run_10", "..."]
}
```

Case IDs match on-disk directory names and are sorted numerically by run number.

## References

- [1] DrivAerML dataset. <https://huggingface.co/datasets/neashton/drivaerml>.
- [2] DrivAerML paper. <https://arxiv.org/abs/2408.11969>.
- [3] Noether DrivAerML split. <https://github.com/Emmi-AI/noether/blob/main/src/noether/data/datasets/cfd/caeml/drivaerml/split.py>.
- [4] PhysicsNeMo DrivAerNet/DrivAerML-style split loader. [https://docs.nvidia.com/deeplearning/physicsnemo/physicsnemo-core/\\_modules/physicsnemo/datapipes/gnn/drivaernet\\_dataset.html](https://docs.nvidia.com/deeplearning/physicsnemo/physicsnemo-core/_modules/physicsnemo/datapipes/gnn/drivaernet_dataset.html).